

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 05 | Elementary Sorting: Bubble Sort, Selection Sort and Insertion Sort

Week 5 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 05 at a glance

Item	Details
Lab title	Elementary Sorting: Bubble Sort, Selection Sort and Insertion Sort
Week	5 of 16
Prerequisites	Lab 03 (Arrays), Lab 04 (Searching)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab05_Instructor.ipynb , Lab05_Student.ipynb
Next lab	Lab 06: Recursion and Divide & Conquer

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain why sorting is important and how it enables fast searching.
2. Implement **Bubble Sort** (with early stopping), **Selection Sort** and **Insertion Sort**.
3. Trace each algorithm by hand with a **pass-by-pass dry run**.
4. Count comparisons and swaps and derive the **best, average and worst case** complexity.
5. Explain **stability** and **in-place** sorting with examples.
6. Sort records (e.g., students) by one or more keys using Python's `sorted()` and `key=`.
7. Choose a suitable elementary sort for small or nearly-sorted data.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement elementary sorting algorithms	Apply (C3)
CLO-3	Analyse and compare sorting algorithms	Analyse (C4)

2. Motivation: Order Makes Everything Faster

In Lab 04 you saw that binary search needs only 20 steps for a million items, **but only if the data is sorted**. Sorting is the step that makes fast searching, ranking and analysis possible. It is so important that some estimates say a noticeable share of all computing time in history has been spent on sorting.

2.1 Sorting in Daily Life

Situation	Sorted by
University merit list	Aggregate marks (descending)
Cricket batting averages / PSL points table	Points, then net run rate
Daraz: "Price: Low to High"	Price
Contacts in your phone	Name (alphabetical)
Playing cards in your hand	You naturally use Insertion Sort!
Students standing in a line by height (PT period)	Height

2.2 Sorting in Industry

- Databases sort data for **ORDER BY**, for building indexes and for joining tables.
- E-commerce and search engines **rank** results; ranking is sorting by relevance score.
- Python's built-in sort (**Timsort**) and C++'s **std::sort** (**Introsort**) both use **Insertion Sort** for small pieces of data, because it is very fast on small or nearly-sorted inputs. So the simple algorithm you learn today runs inside real industrial software.

2.3 Sorting in Research

- **Machine learning:** decision trees sort feature values to find the best split; evaluation metrics like ROC-AUC sort predictions by score.
- **Information retrieval:** “learning to rank” research is about sorting documents by relevance.
- **GIS:** spatial data is often sorted along space-filling curves (e.g., Z-order) for fast queries.

2.4 Why It Matters in 2026–2030

Learning to think like an algorithm designer

You will rarely write bubble sort in a real job; you will call `sorted()`. But elementary sorts teach the core skills every engineer needs: **loop invariants**, **dry runs**, **counting operations** and **comparing trade-offs**. These skills let you understand faster algorithms (Lab 07) and evaluate any code, whether written by a human or an AI.

3. Part A: Basic Terms

Term	Meaning	Example
Ascending / descending	Smallest first / largest first	[1, 3, 7] / [7, 3, 1]
Comparison	Checking which of two elements is bigger	<code>a[j] > a[j+1]</code>
Swap	Exchanging two elements	<code>a[i], a[j] = a[j], a[i]</code>

Term	Meaning	Example
Pass	One complete scan over the (unsorted part of the) array	-
In-place	Uses only $O(1)$ extra memory	Bubble, Selection, Insertion
Stable	Equal elements keep their original relative order	Two students with 80 marks stay in roll-number order

4. Part B: Bubble Sort

Idea: repeatedly compare **adjacent** elements and swap them if they are in the wrong order. After each pass, the **largest remaining element “bubbles up”** to its correct position at the end.

Pseudocode

```

ALGORITHM BubbleSort(A)
  FOR i ← 0 TO n - 2 DO                                // pass number
    swapped ← FALSE
    FOR j ← 0 TO n - 2 - i DO                            // last i elements already
      sorted
        IF A[j] > A[j + 1] THEN
          swap A[j], A[j + 1] ; swapped ← TRUE
    IF swapped = FALSE THEN BREAK                      // already sorted: stop early

```

Dry run on [5, 1, 4, 2, 8]

Pass	Array after the pass	Fixed at the end
1	[1, 4, 2, 5, 8]	8
2	[1, 2, 4, 5, 8]	5, 8
3	[1, 2, 4, 5, 8] (no swaps)	Stop early 

```
def bubble_sort(arr):
    a = arr[:]                # work on a copy
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        print(f"Pass {i + 1}: {a}")
        if not swapped:
            break
    return a

bubble_sort([5, 1, 4, 2, 8])
```

5. Part C: Selection Sort

Idea: find the **minimum** element in the unsorted part and **put it at the front** (swap it with the first unsorted element). Repeat for the rest. Like choosing the shortest student and sending them to the front of the line, again and again.

Pseudocode

```
ALGORITHM SelectionSort(A)
  FOR i ← 0 TO n - 2 DO
    min_index ← i
    FOR j ← i + 1 TO n - 1 DO
      IF A[j] < A[min_index] THEN min_index ← j
    swap A[i], A[min_index]
```

Dry run on [64, 25, 12, 22, 11]

i	Minimum found	Array after swap
0	11	[11, 25, 12, 22, 64]
1	12	[11, 12, 25, 22, 64]
2	22	[11, 12, 22, 25, 64]
3	25	[11, 12, 22, 25, 64]

```
def selection_sort(arr):
    a = arr[:]
    n = len(a)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if a[j] < a[min_index]:
                min_index = j
        a[i], a[min_index] = a[min_index], a[i]
        print(f"i = {i}: {a}")
    return a

selection_sort([64, 25, 12, 22, 11])
```



Selection sort's special property

It makes **at most $n - 1$ swaps**, the fewest of all three. This is useful when writing to memory is expensive (e.g., flash memory or EEPROM in embedded devices).



6. Part D: Insertion Sort

Idea: build the sorted part one element at a time. Take the next element (the **key**), **shift** bigger elements of the sorted part one place to the right, and **insert** the key into the gap. This is exactly how you arrange playing cards in your hand.

Pseudocode

```
ALGORITHM InsertionSort(A)
  FOR i ← 1 TO n - 1 DO
    key ← A[i] ; j ← i - 1
    WHILE j ≥ 0 AND A[j] > key DO
      A[j + 1] ← A[j]          // shift right
      j ← j - 1
    A[j + 1] ← key             // insert
```

Dry run on [12, 11, 13, 5, 6]

i	key	Array after inserting key	Sorted part
1	11	[11, 12, 13, 5, 6]	[11, 12]
2	13	[11, 12, 13, 5, 6]	[11, 12, 13]
3	5	[5, 11, 12, 13, 6]	[5, 11, 12, 13]
4	6	[5, 6, 11, 12, 13]	all

```
def insertion_sort(arr):  
    a = arr[:]  # Create a copy of the array  
    for i in range(1, len(a)):  
        key = a[i]  
        j = i - 1  
        while j >= 0 and a[j] > key:  
            a[j + 1] = a[j]      # shift  
            j -= 1  
        a[j + 1] = key          # insert  
        print(f"i = {i}, key = {key:>2}: {a}")  
    return a  
  
insertion_sort([12, 11, 13, 5, 6])
```



7. Part E: Analysis and Comparison

7.1 Counting the Work

For bubble and selection sort, the inner loop runs $(n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2}$ times in the worst case, which is $\mathbf{O(n^2)}$. For insertion sort, the worst case (reverse-sorted input) also shifts $n(n-1)/2$ times, but on **already sorted** input the while loop never runs, so it is $\mathbf{O(n)}$.

Algorithm	Best	Average	Worst	Space	Stable?	Swaps (worst)
Bubble (early stop)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	$O(n)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	$O(n^2)$ shifts

```
import random

def count_ops(sort_name, arr):
    a = arr[:]; n = len(a); comps = writes = 0
    if sort_name == "bubble":
        for i in range(n - 1):
            swapped = False
            for j in range(n - 1 - i):
                comps += 1
                if a[j] > a[j + 1]:
                    a[j], a[j + 1] = a[j + 1], a[j]; writes += 1; swapped = True
            if not swapped:
                break
    elif sort_name == "selection":
        for i in range(n - 1):
            m = i
            for j in range(i + 1, n):
                comps += 1
                if a[j] < a[m]:
                    m = j
            if m != i:
                a[i], a[m] = a[m], a[i]; writes += 1
    else:
        for i in range(1, n):
            key, j = a[i], i - 1
            while j >= 0:
                comps += 1
                if a[j] <= key:
                    break
                a[j + 1] = a[j]; writes += 1; j -= 1
            a[j + 1] = key
    assert a == sorted(arr)
    return comps, writes

n = 500
inputs = {"random": random.sample(range(n), n), "sorted": list(range(n)), "reversed": list(range(n, 0, -1))}
print(f"{'input':<9} {'algorithm':<10} {'comparisons':>12} {'swaps/shifts':>13}")
for name, data in inputs.items():
    for alg in ["bubble", "selection", "insertion"]:
        c, w = count_ops(alg, data)
        print(f"{'name':<9} {'alg':<10} {'c':>12,} {'w':>13,}")
```

Look at the “sorted” rows: bubble and insertion do only $n - 1$ comparisons, while selection still does $n(n-1)/2$. That is why insertion sort is the champion for nearly-sorted data.

7.2 Stability: Why It Matters

Suppose the student list is already in roll-number order, and we sort by marks. A **stable** sort keeps students with equal marks in roll-number order; an unstable sort may mix them up.

```
students = [("Ali", 80), ("Sara", 95), ("Hamza", 80), ("Zain", 70)]

def selection_sort_by_marks(recs):
    a = recs[:]
    for i in range(len(a) - 1):
        m = i
        for j in range(i + 1, len(a)):
            if a[j][1] < a[m][1]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a

print("Selection (unstable):", selection_sort_by_marks(students))
print("sorted() (stable) :", sorted(students, key=lambda s: s[1]))
```

Selection sort moved Hamza **before** Ali (the long swap jumped over Ali), even though Ali came first originally. Python’s `sorted()` is guaranteed stable.

8. Part F: Sorting in Real Python Code

In real projects, use the built-in tools. `sorted(list)` returns a new sorted list; `list.sort()` sorts in place. Both are **$O(n \log n)$** and stable. The `key` parameter tells Python **what to sort by**.

```
students = [
    {"name": "Ali", "cgpa": 3.4, "age": 20},
    {"name": "Sara", "cgpa": 3.9, "age": 21},
    {"name": "Hamza", "cgpa": 3.4, "age": 19},
]
by_cgpa = sorted(students, key=lambda s: s["cgpa"], reverse=True)
print([s["name"] for s in by_cgpa])

# Multiple keys: CGPA high to low, then age low to high
ranked = sorted(students, key=lambda s: (-s["cgpa"], s["age"]))
print([s["name"] for s in ranked])

words = ["banana", "Apple", "cherry"]
print(sorted(words), sorted(words, key=str.lower), sorted(words, key=len))
```

9. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void insertionSort(vector<int>& a) {
    for (size_t i = 1; i < a.size(); i++) {
        int key = a[i];
        int j = (int)i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

int main() {
    vector<int> a = {12, 11, 13, 5, 6};
    insertionSort(a);
    for (int x : a) cout << x << " ";
    cout << endl;

    vector<pair<string,int>> s = {"Ali",80},{"Sara",95},{"Hamza",80}};
    stable_sort(s.begin(), s.end(),
        [](auto& x, auto& y){ return x.second > y.second; });
    for (auto& p : s) cout << p.first << " " << p.second << endl;
}
```

10. Common Mistakes

Mistake	Effect	Fix
Inner loop <code>range(n)</code> in bubble sort	<code>a[j+1]</code> IndexError	Use <code>range(n - 1 - i)</code>
Swapping inside selection sort's inner loop	Becomes a slower, different algorithm	Swap once after finding the minimum
<code>while a[j] > key and j >= 0</code>	Checks <code>a[-1]</code> when <code>j = -1</code>	Check <code>j >= 0</code> first
<code>result = list.sort()</code>	result is <code>None</code>	Use <code>sorted(list)</code> or sort then use the list

Mistake	Effect	Fix
Assuming every sort is stable	Wrong order of equal records	Use a stable sort or add a tie-break key

11. Lab Tasks

Solve all tasks in `Lab05_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Bubble Sort with Statistics

Write `bubble_sort_stats(arr)` that returns `(sorted_list, passes, swaps)` using bubble sort with early stopping. Do not modify the input list.

```
bubble_sort_stats([5, 1, 4, 2, 8]) → ([1, 2, 4, 5, 8], 3, 4)
```

Task 2 Merit List with Selection Sort

Write `merit_list(records)` that sorts `(name, marks)` tuples by marks in **descending** order using **your own selection sort** (no `sorted/sort`). Return the list of names.

```
[("Ali", 72), ("Sara", 91), ("Usman", 85)] → ["Sara", "Usman", "Ali"]
```

Task 3 Stable Insertion Sort for Records

Write `insertion_sort_by(records, key)` that sorts a list of dictionaries by `record[key]` in ascending order using insertion sort. It must be **stable**. Return a new list.

```
by "marks": [{"n": "Ali", "marks": 80}, {"n": "Sara", "marks": 70},
{"n": "Hamza", "marks": 80}]
→ Sara(70), Ali(80), Hamza(80)    # Ali stays before Hamza
```

Task 4 Sort Colours (Dutch National Flag)

An array contains only 0s, 1s and 2s (e.g., red, white, blue balls). Sort it **in one pass** and **in place** without any sorting function.

Hint: keep three pointers `low`, `mid`, `high`. 0 → swap with low; 1 → move on; 2 → swap with high.

```
[2, 0, 2, 1, 1, 0] → [0, 0, 1, 1, 2, 2]
```

Task 5 🟡 Rank Students by Multiple Keys

Using Python's `sorted()` with a **key**, rank students by **CGPA (high → low)**, then **attendance (high → low)**, then **name (A → Z)**. Return the list of names.

```
[("Bilal",3.5,90), ("Ayesha",3.8,85), ("Ahmad",3.5,95), ("Zara",3.5,90)]
→ ["Ayesha", "Ahmad", "Bilal", "Zara"]
```

Task 6 🔴 Count Inversions (How Unsorted Is It?)

An **inversion** is a pair (i, j) with $i < j$ and $a[i] > a[j]$. The number of inversions measures how far an array is from sorted, and it is exactly the number of **shifts** insertion sort performs.

Write `count_inversions(arr)` by instrumenting insertion sort (count the shifts). Then verify it against a brute-force $O(n^2)$ double loop.

```
[2, 4, 1, 3, 5] → 3    # (2,1), (4,1), (4,3)
```

Task 7 🔴 Nearly Sorted Data

A sensor log is **almost sorted** by time: each reading is at most **k** positions away from its correct position. Show experimentally that insertion sort is fast here.

Write `make_nearly_sorted(n, k)` that returns a list of $0..n-1$ where elements are shuffled only within blocks of size **k**, and `insertion_shifts(arr)` that returns the number of shifts. Show that $\text{shifts} \leq n \cdot k$, far less than $n^2/4$ for random data.

🏆 12. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab05** folder.

#	Problem	Level	Concept practised
1051	Height Checker	Easy	Compare with sorted order
977	Squares of a Sorted Array	Easy	Sorting / two pointers
242	Valid Anagram	Easy	Sort characters
1122	Relative Sort Array	Easy	Custom sort key
75	Sort Colors	Medium	Dutch national flag (Task 4)
912	Sort an Array	Medium	Implement a sort (try insertion, see the time limit!)



13. Home Task

1. Finish all tasks and push `Lab05_Student.ipynb` to GitHub.
2. Implement **Cocktail Shaker Sort** (bubble sort that goes left-to-right then right-to-left) and compare its passes with bubble sort.
3. Modify insertion sort to use **binary search** to find the insert position. Does it reduce comparisons? Does it reduce shifts? Explain.
4. **Mini-project: PSL Points Table.** Store at least 6 teams with points, net run rate and wins. Sort by points, then NRR, then wins using (a) your own insertion sort with a comparison function and (b) `sorted()` with a tuple key. Print a formatted table.
5. Solve at least 3 LeetCode problems from the list.

? 14. Viva Questions

1. Explain bubble sort in your own words. Why is it called “bubble”?
2. How does early stopping make bubble sort $O(n)$ in the best case?
3. Why does selection sort always take $O(n^2)$ comparisons?
4. Which elementary sort makes the fewest swaps? When is that useful?
5. Why is insertion sort good for nearly-sorted data?
6. What is a stable sort? Give an example where stability matters.
7. Is selection sort stable? Show a counter-example.
8. What does “in-place” mean?
9. What is the difference between `sorted(a)` and `a.sort()`?
10. Why do Timsort and Introsort use insertion sort internally?



15. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

16. References and Further Learning

- Sorting HOW TO (Python docs): docs.python.org/3/howto/sorting.html
- Animated sorting algorithms: visualgo.net/en/sorting
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.